

EAP — Estrutura Analítica do Projeto · Agendamento

Clínica Vitalis

Aluno: Roberto Vital de Araujo · **Disciplina:** Programação e Desenvolvimento Web · **Professor:** Everson Andrade dos Reis · **CEUB**

O que é uma EAP

A Estrutura Analítica do Projeto (EAP; em inglês, *WBS*) é a decomposição hierárquica e **orientada a entregas** de todo o trabalho do projeto. Ela parte do resultado final (nível 1) e desce em partes cada vez menores, até os **pacotes de trabalho** — o menor nível, que pode ser estimado, atribuído e controlado.

Regras que esta EAP segue:

- **Regra dos 100%:** a soma dos itens-filhos representa exatamente o escopo do item-pai — nada a mais, nada a menos.
- **Orientada a entregas** (substantivos: "Validação de CPF"), não a tarefas/ações (verbos).
- **Decomposição por níveis** com código numérico (1 → 1.1 → 1.1.1).
- Cada pacote de trabalho é descrito no **dicionário da EAP** (adiante).

A EAP

1 Sistema de Agendamento Clínica Vitalis (projeto)

1.1 Gestão e planejamento

- 1.1.1 Escopo e requisitos do agendamento
- 1.1.2 Documentação (EAP, README, PDF de entrega)

1.2 Modelagem de dados

- 1.2.1 Especialidades e profissionais (profissionais.json)
- 1.2.2 Regras de disponibilidade (dias e horários por profissional)
- 1.2.3 Armazenamento de agendamentos (agendamentos.json)

1.3 Backend (API em Flask)

- 1.3.1 Leitura dos dados locais (JSON)
- 1.3.2 Rotas de especialidades e profissionais
- 1.3.3 Cálculo de datas e horários disponíveis
- 1.3.4 Criação de agendamento com validação de CPF
- 1.3.5 Consulta de agendamentos por CPF
- 1.3.6 Cancelamento de agendamento

1.4 Frontend

- 1.4.1 Fluxo em etapas (especialidade → profissional → data → dados)
- 1.4.2 Formulário com máscara e validação de CPF
- 1.4.3 Aba "Meus agendamentos" (consulta e cancelamento)
- 1.4.4 Identidade visual Vitalis e responsividade

1.5 Qualidade e testes

- 1.5.1 Testes das rotas da API
- 1.5.2 Teste do ciclo completo (criar, consultar, cancelar)
- 1.5.3 Testes de validação (CPF e concorrência de horário)

1.6 Publicação e entregáveis

- 1.6.1 Repositório no GitHub (pasta agendamento/)
- 1.6.2 README com instruções de execução
- 1.6.3 Vídeo de apresentação
- 1.6.4 PDF de entrega

Dicionário da EAP

Descrição de cada pacote de trabalho (nível folha).

Código	Pacote de trabalho	Entrega / resultado
1.1.1	Escopo e requisitos	Funcionalidades definidas: escolher especialidade e profissional, ver datas, agendar com nome e CPF, consultar e cancelar
1.1.2	Documentação	EAP, README e este PDF de entrega
1.2.1	Especialidades e profissionais	profissionais.json com 5 especialidades e 6 profissionais (compartilhado com o Projeto 02)
1.2.2	Regras de disponibilidade	Dias da semana e horários atendidos por cada profissional
1.2.3	Armazenamento	agendamentos.json — arquivo local que guarda cada marcação
1.3.1	Leitura de dados locais	Backend lê o JSON local a cada requisição
1.3.2	Rotas de especialidades/profissionais	/api/especialidades e /api/profissionais (com filtro por especialidade)
1.3.3	Disponibilidade	/api/disponibilidade — datas e horários livres nos próximos 21 dias
1.3.4	Criar agendamento	POST valida nome e CPF (dígitos verificadores) e grava a marcação
1.3.5	Consultar por CPF	/api/agendamentos?cpf= retorna as marcações do paciente
1.3.6	Cancelar	Marca como cancelado e devolve o horário à disponibilidade
1.4.1	Fluxo em etapas	Interface que avança especialidade → profissional → data → dados
1.4.2	Formulário com CPF	Campos de nome e CPF, com máscara e validação no envio
1.4.3	Meus agendamentos	Aba que consulta por CPF e permite cancelar
1.4.4	Identidade visual	Mesma paleta e tipografia da Vitalis; responsivo
1.5.1	Testes das rotas	Todas as rotas verificadas, inclusive erros 400/404/409
1.5.2	Ciclo completo	Criar, consultar e cancelar testados de ponta a ponta
1.5.3	Validações	CPF válido/inválido e horário duplicado (conflito 409)
1.6.1	Repositório GitHub	Código versionado na pasta agendamento/
1.6.2	README	Instruções de execução do sistema
1.6.3	Vídeo	Apresentação do sistema funcionando
1.6.4	PDF de entrega	Documento com descrição e links

Premissas e exclusões

- **Premissas:** aplicação executada localmente (backend em Python + Flask); armazenamento em arquivo JSON local; dados fictícios; CPF validado apenas quanto aos dígitos verificadores

(nenhum dado real).

- **Fora do escopo:** pagamento, login de usuários, envio de e-mail/SMS, banco de dados e hospedagem do backend em servidor online (o GitHub Pages hospeda apenas a landing page estática do Projeto 01).